

RapidPnR: Accelerating the Physical Design for FPGAs via Design-Level Parallelism

Abstract—The runtime of physical design has become a critical issue for FPGA development as the scale and complexity of circuit designs surge with the increasing logic capacity of FPGA devices. The time-consuming process of physical design significantly extends the cycle of design iteration, which heavily impacts the efficiency of debugging and architecture optimization of circuit designs. To address this issue, this work proposes a generic, fully-automated and split-and-parallel physical design flow to accelerate the deployment of large-scale circuits on FPGAs. Specifically, our flow automatically partitions the synthesized netlist into multiple smaller pieces, performs parallel physical design of each piece, and then merges them into the complete design. Evaluated on a set of real circuit benchmarks, our flow reduces the runtime by more than 50% and ensures nearly the same design frequency compared to the physical design flow provided by the commercial tool Vivado.

Index Terms—FPGA, physical design, divide-and-conquer, parallelism, acceleration

I. INTRODUCTION

As the scale and complexity of designs continue to grow, the runtime issue of physical design on FPGAs has become increasingly significant [9], [10]. Although the logic capacity of FPGAs keeps expanding exponentially with rapid advancements in semiconductor technology, the time-consuming process of physical design heavily impedes the deployment of larger circuits on them [10]. Typically, the development of FPGA systems is an iterative process, which involves multiple rounds of architecture design, RTL implementation, logic synthesis, physical design and system testing. Slow physical design processes significantly extend the cycle of design iteration, which affects the efficiency of debugging and performance optimization of target designs.

In recent years, numerous efforts have been made to accelerate the physical design for FPGAs, and the methodologies can generally be classified into two categories, algorithm-level parallelism and design-level parallelism. Algorithm-level parallelism first decomposes key physical design algorithms, such as placement [1]–[3], routing [4]–[7] and static timing analysis (STA) [8], into multiple independent tasks, then executes these tasks in parallel on GPUs [1], [2], [7] or multi-core CPUs [3], [4]. Although the acceleration is enhanced by the increasing parallel computing capabilities of GPUs or CPUs, the inherently sequential nature of most physical design algorithms limits further speedup. Besides, most of these works don't consider timing optimization, which accounts for most runtime of high-performance physical design and is difficult to parallelize [9]. In contrast, design-level parallelism directly partitions the original design into multiple

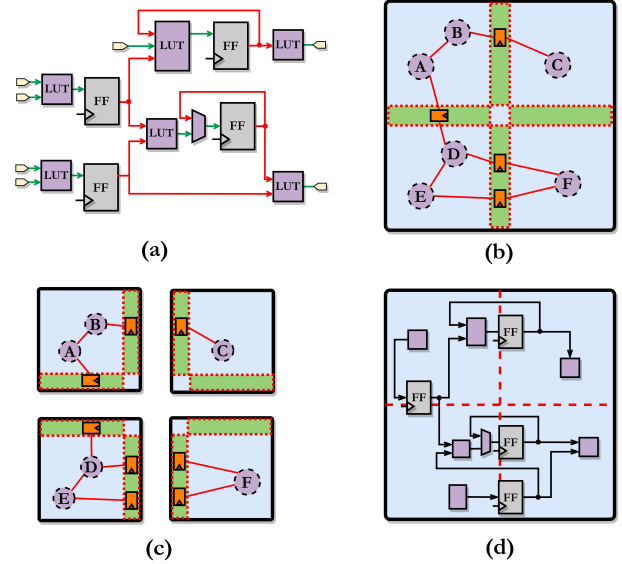


Fig. 1. An illustration of proposed split-and-parallel physical design flow: (a) the synthesized netlist produced by vendor tools; (b) transform the original netlist into an abstracted hypergraph, map this hypergraph into 2x2 islands, and extract and assign the source cell of each inter-island net to the boundary region; (c) split and parallel physical design of each island; (d) merge islands into the complete design.

small sub-designs, then performs physical design on each sub-design concurrently, and finally merge the results into a complete placed and routed design [9]–[13]. Xiao et al. [10]–[13] partitions an entire FPGA into multiple disjoint blocks connected with a lightweight Network on Chip (NoC) or direct wires. Then they distribute operators into these blocks in High-Level Synthesis (HLS) designs and implement each block in parallel. Rapidstream [9] first partitions dataflow-based HLS designs into a mesh of disjoint islands, then insert pipelines into inter-island connections, and finally perform the physical design of each island concurrently.

The state-of-the-art design-level parallelism works only focus on HLS designs with the dataflow architecture [9], [12]. Besides, they perform partitioning at the architectural level of design, without considering netlist-level information that is highly related with physical design, such as interconnections between primitive cells, accurate resource utilization, and delay of timing paths. Therefore these approaches can't be applied to generic scenarios where physical design starts directly from the synthesized netlist regardless of detailed circuit architecture. In this work, we explore the idea of design-level parallelism at the netlist level. We try to answer this question: can we directly partition the synthesized netlist and

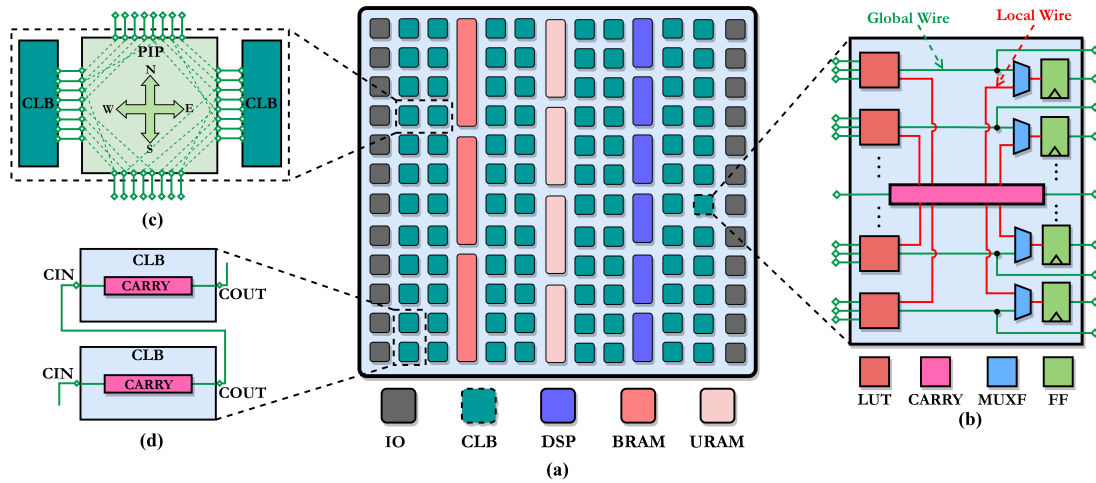


Fig. 2. A simplified architecture of AMD-Xilinx Ultrascale+ FPGA: (a) Primitive Organization: CLB, DSP, BRAM and URAM are arranged in a columnar way; (b) Internal structure of CLB site; (c) General inter-site routing resources consisting of PIP and wires connecting neighboring sites; (d) Dedicated inter-site wires connecting CIN and COUT ports of CARRYs in the vertically adjacent CLB.

then perform physical design of each sub-netlist in parallel?

Two critical issues arise when adopting design-level parallelism at the netlist level. The first issue is how to ensure the timing quality of inter-partition timing paths, while the second relates to routing and merging inter-partition nets, as well as the placement of cells incident to these nets. To address these two issues, we propose a fully automated, split-and-parallel and high-performance physical design flow to accelerate the placement and routing of the synthesized netlist. More specifically, our flow consists of three main stages, netlist abstraction, design partition and parallel placement and routing (PnR) & merging, as illustrated in Fig. 1 and Fig. 3. The first stage transforms the synthesized netlist into an abstract hypergraph. In this stage, we try to cluster combinational nets and cells and only expose registered nets in the abstract hypergraph to achieve timing isolation after design partition. In the second stage, the entire FPGA is divided into disjoint islands using a grid. Then we develop an efficient two-stage algorithm to distribute abstract nodes to these islands, ensuring multiple utilization constraints and minimizing the number of inter-island nets. To guide the placement and routing of inter-island nets, we adopt the idea of anchor registers [9] by extracting source cell of each inter-island net and assigning it to boundary region. Then we perform physical design of each island along with its neighboring anchor region in parallel and merge them into the complete design.

The main contributions of our work can be summarized as follows

- To the best of our knowledge, our work is the first to extend the methodology of design-level parallelism to more general and practical use cases where physical design starts from the synthesized netlist.
- To ensure the legality and performance of the merged design, we propose a series of techniques, such as netlist abstraction, two-stage island placement, anchor cell ex-

traction and external delay prediction, and integrate them into an automated, split-and-parallel physical design flow.

- Evaluated on a set of diverse benchmark netlists, our flow speeds up the physical design by 1.6x–2.5x with an average frequency degradation of only 6% compared to the commercial tool Vivado [29].

II. PRELIMINARIES

This section highlights key characteristics of the employed FPGA architecture that should be carefully considered in the physical design. Additionally, a brief introduction of FPGA physical design is also provided. In this work, our flow targets the AMD-Xilinx Ultrascale+ FPGA [14], which is one of the most widely used FPGA families nowadays. A simplified Ultrascale+ architecture is shown in Fig. 2.

A. Primitive Organization

Under the Xilinx Ultrascale+ architecture, the FPGA device primarily consists of seven types of primitive instances [17], including Block RAM (BRAM), Digital Signal Processing Unit (DSP), UltraRAM (URAM), Lookup Table (LUT), Carry Chain (CARRY), Multiplexer (MUXF) and Flip Flop (FF). These primitives are organized hierarchically within the device. At the lowest level of hierarchy, multiple LUTs, MUXFs, FFs and one CARRY are packed into a Configurable Logic Block (CLB) [15]. In addition to these primitive instances, each CLB site also contains site pins (which act as interface of CLB to externals), and site wires (which connect primitive instances and site pins). At the next level of hierarchy, sites of CLB, BRAM, DSP and URAM are arranged in columns within the device layout, with each column occupied by one type of primitive. Other hierarchical levels in the Ultrascale+ architecture include the clock region (which consists of arrays of sites) and the super logic region (which arranges arrays of clock regions into a chiplet for multi-die FPGAs) [21].

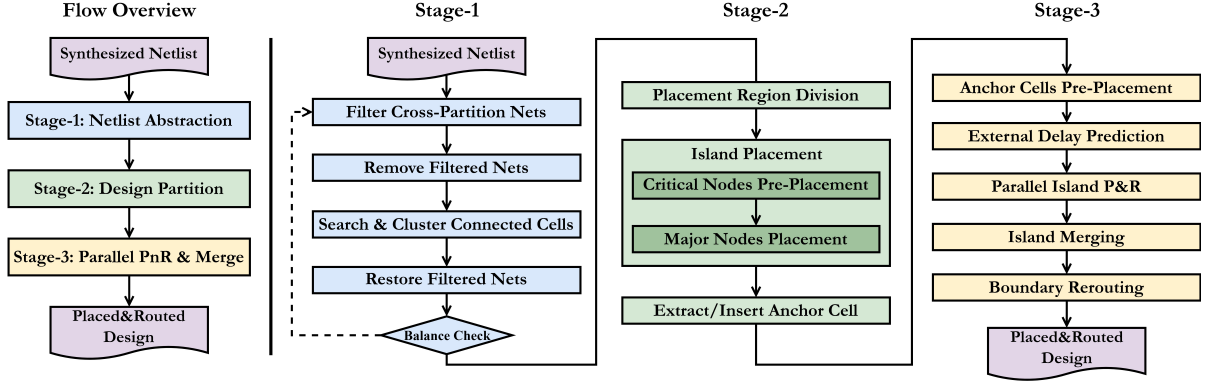


Fig. 3. The proposed split-and-parallel physical design flow for FPGAs.

B. Routing Architecture

The interconnections between primitive instances in the Ultrascale+ FPGA are established through two kinds of routing resources, including intra-site and inter-site. Inter-site routing resources can be further categorized into general interconnections and dedicated interconnections. General inter-site routing resources are organized in interconnect (INT) tiles [21]. They connect pins of CLB, BRAM, URAM or DSP sites using switchboxes, also known as programmable interconnect point (PIP), and wires of different lengths. Specifically, each site is directly connected with the PIP in its neighboring INT tile and then this PIP are connected with others through wire segments in four cardinal directions. Dedicated inter-site routing resources are hardwired connections between specific pins of adjacent sites in one column [26]. These wires are usually used for the fast transmission of chaining signals between cascaded CARRYs, BRAMs or DSPs. For example, the COUT pin of lower CARRY and CIN pin of upper CARRY are connected directly through a wire without going through any switchboxes as shown in Fig. 2 (d). Intra-site routing resources consist of wires connecting primitive instances and site pins within a CLB site. These wires can be categorized into local or global, depending on whether they connect sites pin and is able to access inter-site routing resources. For instance, as shown in Fig. 2 (b), the connection between CARRY and LUT, or MUXF and FF, are restricted to the specific wire within one CLB site. While the input and output pins of LUTs or FFs can access inter-site wires through site pins to connect to FFs and LUTs in other CLB sites.

C. Physical Design

Similar to the ASIC design flow, the deployment of RTL designs on FPGAs primarily involves two phases: synthesis and physical design. The synthesis process transforms RTL codes into a netlist consisting of primitive cells and nets connecting them. The physical design is responsible for implementing the synthesized netlist using primitive instances and routing resources available in the FPGA device. In detail, the physical design can be further divided into placement and routing. Placement maps cells in the netlist to primitive instances

of the same type in the FPGA device while optimizing the total wirelength, timing quality and routability. Given sites of primitive cells, routing establishes the physical connections of nets in the netlist using prefabricated wires and PIPs.

The characteristics of routing architecture in the Ultrascale+ FPGA impose certain constraints on design partitioning considering the legality of physical design. Specifically, it's undesirable that nets restricted to local intra-site wires or dedicated inter-site wires are cut during netlist partitioning. For example, if two cascaded CARRYs are partitioned into different columns in the device, the carry signal net fails to be routed. Similarly, routing errors occur if connected MUXF and FF are partitioned into different CLB sites.

TABLE I
TERMINOLOGY AND NOTATION

Notation	Description
m	The number of columns in the island grid
n	The number of rows in the island grid
R	Set of critical resources: BRAM, DSP and URAM
$G'(V', E')$	Partial hypergraph of critical nodes and their neighbors
(x_v, y_v)	Location of node v in the island grid
I	Set of disjoint islands
(x_i, y_i)	Location of island i in the grid
$r(i)$	Amount Limit of resource $r \in R$ on island $i \in I$
$r(v)$	Amount of resource $r \in R$ in node $v \in V'$
α	Weighting factor of predicted net delay

III. THE PROPOSED SPLIT-AND-PARALLEL FLOW

The overview of the proposed split-and-parallel physical design flow is illustrated in Fig. 3. The entire flow mainly consists of three stages, netlist abstraction, design partition and parallel PnR & Merge. Netlist abstraction transforms the synthesized netlist into an abstract hypergraph with clustered primitives as nodes. During design partition stage, the abstract hypergraph is split into multiple parts and each part is assigned with a disjoint and well-sized placement region. After design partition, each sub-design with both sub-netlist and constrained placement region is placed and routed parallelly and then merged together into the complete design. The detailed description of each stage is provided in the following sections.

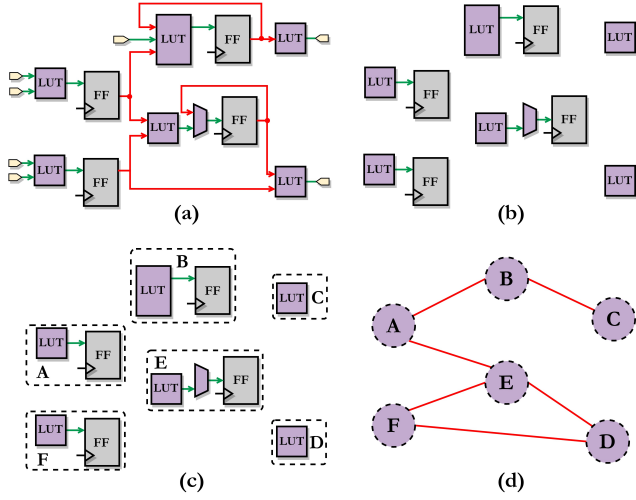


Fig. 4. The process of netlist abstraction on a simple netlist: (a) filtered netlist with hidden nets marked as green and removed nets marked as red; (b) the remaining netlist after removal of red nets; (c) abstract vertices with clustered connected instances; (d) complete abstract hypergraph.

A. Netlist Abstraction

The intuition behind netlist abstraction is to hide certain nets in the synthesized netlist through cell clustering. Actually it's aimed to impose restrictions on the later stage of design partition, considering both legality and design performance. For legality of the partitioned design, as discussed in Section II-C, it's critical that nets restricted to dedicated inter-site or local intra-site wires remain uncut to avoid routing errors. With regard to design performance, it is desirable for all timing paths to reside within the same partition, thereby eliminating potential timing violations associated with inter-partition paths. To achieve this, we need to ensure that there are no cuts on any internal nets of each timing path. In other words, only nets driven by registers are allowed to be cut, which ensures that there is always a register at any junction of different partitions for timing isolation. The detailed implementation of netlist abstraction is described in Algorithm 1. And Fig. 4 shows the process of netlist abstraction on a simplified circuit netlist.

The first step of netlist abstraction is to filter out inappropriate nets that should be concealed in the abstract hypergraph. And for different designs, we adopt different filtering schemes. First of all, nets restricted to dedicated and local routing resources are filtered out for all designs. Based on this fundamental criterion, any nets except for those driven by FF are also filtered out. But experimental results in Section IV-C show that not all circuits get nodes of balanced sizes in the abstract hypergraph if we try to conceal all combinational nets. For some circuits, there exist extremely large vertices in the abstract hypergraph, which makes it impossible to get multiple balanced sub-designs in the stage of design partition. So netlist abstraction process is performed iteratively by gradually relaxing net filtering criteria. If we can't get balanced abstract hypergraph using aforementioned filtering criterion, we can relax it by allowing nets driven by LUT to be exposed to

design partition.

To identify whether a net satisfies certain criteria, we traverse all incident primitive cells of this net and see whether it connects specific ports of specific cells. For example, we can identify that a net is restricted to dedicated wires for carry signals if it connects the COUT and CIN ports of two cells of CARRY.

Algorithm 1: Netlist Abstraction

Input: Original netlist hypergraph $G(E, V)$
Output: Abstracted netlist hypergraph $\hat{G}(\hat{E}, \hat{V})$

```

1  $isEdgeVisited(e) \leftarrow false$  for  $e \in E$ ;
2  $removedEdges \leftarrow \{\}$ ;
3  $origin2AbstractMap \leftarrow \{\}$ ;
4 foreach edge  $e \in E$  do
5   if not  $isHiddenEdge(e)$  then
6      $isEdgeVisited(e) \leftarrow true$ ;
7      $removedEdges.add(e)$ ;
8 foreach vertex  $v \in V$  do
9   if  $origin2AbstractMap.contains(v)$  then
10     $continue$ ;
11    $queue \leftarrow \{v\}$ ;
12    $\hat{v} \leftarrow \{v\}$ ;
13    $origin2AbstractMap.put(v, \hat{v})$ ;
14   while not  $queue.isEmpty()$  do
15      $v' \leftarrow queue.pop()$ ;
16     foreach net  $e$  incident to  $v'$  do
17       if  $isNetVisited(e)$  then
18          $continue$ ;
19        $isNetVisited(e) \leftarrow true$ ;
20       foreach vertex  $v''$  incident to  $e$  do
21          $\hat{v} \leftarrow \hat{v} \cup v''$ ;
22          $origin2AbstractMap.put(v'', \hat{v})$ ;
23          $queue.push(v'')$ ;
24    $\hat{V} \leftarrow \hat{V} \cup \hat{v}$ ;
25 foreach net  $e \in removedEdges$  do
26    $\hat{e} \leftarrow \{\}$ ;
27   foreach vertex  $v$  incident to  $e$  do
28      $\hat{v} \leftarrow origin2AbstractMap.get(v)$ ;
29      $\hat{e} \leftarrow \hat{e} \cup \hat{v}$ ;
30    $\hat{E} \leftarrow \hat{E} \cup \hat{e}$ ;

```

After filtering out nets to be concealed, we remove all other nets from the original netlist hypergraph. And then we search for connected components on this remaining hypergraph, corresponding to line 8-24 in Algorithm 1. Then all connected cells are clustered into a new node in the abstracted hypergraph. Finally, we traverse each previously removed net, identify all abstract nodes incident to it, and add an edge to connect these nodes in the abstract hypergraph.

B. Design Partition

The stage of design partition is responsible for splitting the original netlist into multiple sub-netlists and allocate a disjoint and appropriately-sized region to each sub-netlist.

1) *Placement Region Division*: In our flow, the entire FPGA device is first divided into disjoint regions using a $m \times n$ grid, which we also refer to as islands. Additionally, a thin region is reserved along each boundary of adjacent islands, which is referred to as anchor region. These anchor regions only consist of several rows or columns of CLB sites. A simplified layout of the divided FPGA device is shown in Fig. 1 (b), in which anchor regions are highlighted in green.

2) *Island Placement*: Then nodes in the abstract hypergraph from the previous stage are distributed to this grid of islands. Specifically, island placement is aimed to balance the size of nodes in each island while minimizing the total number and length of inter-island nets. Besides it must guarantee that there is no over-utilization of each kind of resource on each island. There are two challenges related with island placement, considering the legality and efficiency respectively. For legality, island placement must simultaneously account for utilization constraints of multiple primitive resources, including BRAM, URAM, DSP, LUT, and FF. Additionally, the heterogeneity of each abstract node, which consists of different primitive resources, further increases the complexity of the problem. For efficiency, the number of abstracted nodes varies from thousands to hundreds of thousands depending on different circuits, as shown in Section IV-C. Therefore, we must carefully consider the time complexity of the adopted algorithm to avoid significant runtime overhead on large-scale abstract hypergraphs. Considering these two challenges, we propose a two-stage algorithm to solve island placement, consisting of critical nodes pre-placement and major nodes placement.

In the first stage, we only focus on the placement of nodes containing critical primitive resources, including BRAM, URAM and DSP. Compared to LUT and FF, these resources are much more scarce and prone to result in over-utilization. Due to their scarcity, the number of abstract nodes containing these resources is also relatively small, ranging from dozens to hundreds. Considering the scale of critical nodes, we can solve their placement optimally using Integer Linear Programming (ILP). However, the optimal placement of few critical nodes may not result in the best placement results of the overall abstract hypergraph. So in addition to critical nodes, some heavily connected neighboring nodes are also considered. Specifically, for each node adjacent to critical nodes, a connectivity value is computed, which equals the total weight of edges incident to both this node and any critical nodes. Then all neighboring nodes are ranked based on this connectivity value and top-ranked ones are selected to construct a partial hypergraph along with all critical nodes.

To formulate the ILP-based placement of this partial hypergraph, we introduce the following variables, detailed descriptions of related notations can be found in Table. I.

- Coordinate of each node: (x_v, y_v) for each $v \in V'$
- Assignment of each node to each island: $a_{v,i} \in \{0, 1\}$ for each $v \in V'$ and each $i \in I$

Then the optimization goal is to minimize the total wire-length of partial hypergraph:

$$\sum_{e \in E'} \max_{v \in e} x_v - \min_{v \in e} x_v + \max_{v \in e} y_v - \min_{v \in e} y_v$$

And constraints are defined as follows:

- $0 \leq x_v < m$ and $0 \leq y_v < n$ for each $v \in V'$
- $a_{v,i} = 1$ iff $x_v = x_i$ and $y_v = y_i$
- $\sum_{i \in I} a_{v,i} = 1$ for each $v \in V'$
- $\sum_{v \in V'} a_{v,i} \times r(v) \leq r(i)$ for each $i \in I$ and each $r \in R$

The first constraint ensure that the location of each node is not out of bound. The following two constraints build relation between location variable and assignment variable. The final constraint ensures that there are no over-utilization of each kind of resources.

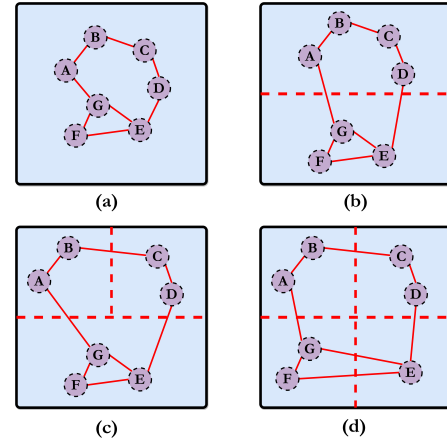


Fig. 5. An illustration of placement of major nodes through iterative bi-partition of the abstract hypergraph.

After the location of critical nodes is determined, the second stage solves the placement of other major nodes in the abstract hypergraph. Considering the runtime efficiency of handling large-scale abstract hypergraph, we adopt the idea of min-cut VLSI placement algorithm [18] [19] to solve this problem. Specifically, the location of abstract nodes is determined through a series of recursive bi-partitions. Fig. 5 illustrates the process of distribute abstract nodes to a 2x2 grid using min-cut island placement. The abstract hypergraph is first partitioned in the dimension of y-axis and then the resulting two smaller partitions are further partitioned in the dimension of x-axis recursively. During the process of partitioning, we need to set appropriate assignment constraints for critical nodes to preserve the placement results from the previous stage. For each iteration of bi-partition, the latest multi-level hypergraph partition algorithm [24] is used to minimize cut size while balancing the size of two blocks.

However, this min-cut placement scheme only considers the optimization of total number of inter-island nets during

partition. Therefore it may cause net crossing boundary more than one time, which may degrade the timing quality of related paths. For example, if nodes of certain net are distributed to diagonal islands, this net needs to cross two boundaries to connect incident nodes. Considering this problem, we also propose an enhanced version of min-cut island placement, which guarantees that each net only spans neighboring islands. To satisfy this constraint, we complement the clustering of nodes incident to cut edges after each partition iteration, thereby hiding these nets and preventing them from being cut in the next iteration. The clustering of nodes incident to cut edges works similar to netlist abstraction introduced in section III-A. Specifically, we remove all edges except those that are cut during partition, search and cluster connected components in the remaining hypergraph, and then restore removed edges to connect clustered nodes.

Similar to netlist abstraction, clustering nodes incident to cut edges may result in extremely large nodes in the clustered hypergraph, which prevents balanced partition in the next iteration. Therefore strict single-boundary constraints on each edge may conflict with the goal of balanced island placement. In our flow, the actual island placement of major nodes is performed iteratively considering the trade-off between the length of inter-island edge and size balancing of each island.

3) *Extract/Insert Anchor Cell*: After the location of each abstract node is determined, we extract the source cell of each inter-island net and assigned it to the corresponding anchor region, as shown in Fig. 1 (b). Additionally, an extra buffer cell is inserted on nets crossing boundaries more than one time so that every crossed boundary has an anchor cell. Specifically, we use LUT1 primitive [17] as the buffer cell in the actual implementation. These anchor cells serve as the interface of neighboring islands and guide the placement and routing of connected cells.

C. Parallel PnR & Merge

Before splitting and implementing each island, we first need to determine and fix the location of anchor cells shared by neighboring islands to ensure their locations remain consistent after parallel physical design. The optimal way to determine the location of anchor cells is to perform the placement of the entire netlist with region constraints on island and anchor cells. However, placing the entire design introduces significant runtime overhead, which conflicts with our goal of accelerating physical design by dividing the original large-scale problem into smaller, more manageable pieces. To reduce runtime overhead while obtaining appropriate locations of anchor cells, we perform placement on the partial netlist consisting of only anchor cells and a fraction of island cells which are directly connected to them. The intuition behind this approach is that locations of anchor cells are primarily influenced by their neighboring cells.

As mentioned in Section III-A, we attempt to cluster all combinational cells and nets in the abstract hypergraph to achieve timing isolation between partitioned islands. However, some designs fail to get balanced partitions if we only

expose register-driven nets in the abstract hypergraph. For these designs, inter-island timing paths inevitably exist, which requires appropriate timing constraints during physical design. In detail, for each island, we need to get the delay of the external segment of each inter-island timing path and set this value as the input/output delay on the corresponding ports. Since the actual delay of timing path segments is unknown before routing, we propose a simple and efficient prediction scheme to estimate its approximate value.

The prediction scheme works as follows: Firstly we model the delay of LUT as 1.0 and the delay of each net as $(1.0 + \alpha \ln(fanout))$. Based on this simplified timing model, we adopt the basic static timing analysis algorithm to estimate the input and output delay of each pin in the netlist. Specifically, we first build a timing graph where each pin is modeled as a timing vertex and connection between each pair of pins is modeled as a timing edge. Then vertices in this graph are traversed in the topological order to calculate their input delay. For each vertex, we set its input delay as the maximum delay from all input edges. And similarly we traverse timing vertices in the reversed topological order again to calculate the output delay of each pin. Then we normalize these approximate input delay D_{in} and output delay D_{out} to actual clock period T_{clk} as follows:

$$D_{in} = \frac{D_{in}}{(D_{out} + D_{in})} \times T_{clk}, D_{out} = \frac{D_{out}}{(D_{out} + D_{in})} \times T_{clk}$$

After fixing anchor cells and setting constraints on inter-partition timing paths, we perform parallel physical design of split islands. When physical design of all islands are completed, we merge them into complete and fully placed and routed design. Because routing resources within anchor regions are shared by neighboring islands, there may exist routing conflictions on them when merging independently-routed islands together. So we perform extra rerouting of conflicted nets to ensure the integrity of merged designs.

IV. RESULTS

A. Implementation Details

Our proposed flow is implemented in Java based on RapidWright platform [20] [21], which provides a gateway to backend tools in the Xilinx FPGA development suite, Vivado. Using RapidWright, we can manipulate synthesized netlist and physical implementation produced by Vivado to construct customized physical design flow. All our experiments are conducted on a Linux server that consists of an AMD EPYC 7543 CPU @ 3.73GHz (32 cores) and 125GB memory. Some implementation details of each stage are outlined as follows:

1) *Netlist Abstraction*: We parse the synthesized netlist produced by Vivado using RapidWright to extract the type of each primitive cell and their interconnection information.

2) *Island Placement*: It's tricky and critical to set the appropriate size of island grid. A grid with higher dimensions splits the design into smaller pieces, which shorten the runtime of implementing individual islands. But it also introduces more inter-island nets, which may increase the runtime of boundary

rerouting and degrade timing quality of the merged design. Considering these issues, we empirically set both the width and height of the island grid to 2, dividing the entire FPGA into four disjoint regions. Our island placement algorithm can also be extended to support other different sizes of grid. We use the open-source ILP solver, Google OR-Tools [31], to solve the ILP problem formulated for pre-placement of critical nodes. And the min-cut island placement is implemented based on the hypergraph partitioner TritonPart [24].

3) *Parallel PnR & Merge*: The physical design of each split island is accomplished using the commercial tool Vivado in parallel. The placement and routing results of each island generated by Vivado are read and merged together through RapidWright. Then we launch Vivado again to reroute conflicted nets within boundary regions.

B. Benchmarks

In our experiments, we target the Xilinx Virtex Ultrascale+vu3p FPGA, which has been widely adopted in recent FPGA EDA contests [23]. This device is the latest single-die FPGA, which consists of 394K LUTs, 788K FFs, 2280 DSPs, 720 BRAMs and 320 URAMs. To evaluate our flow, we collect a set of open-source circuit designs, including corundum [27], blue-rdma [32], nvdla [30], nantucket [34], minimap [28], ispd16-fpga02 [35] and hardcamel-ntt [33]. These designs contain both synthetic netlist and real circuit designs, targeting diverse applications, including networking, deep learning(DL), cryptography and genetics. On the basis of this set of designs, we change configuration parameters of some designs to produce a total of ten benchmark netlists with varying resource utilization. Specifically, all our netlists are synthesized under the out-of-context mode [16], which means that we don't consider the placement and routing of top-level IOs buffers during physical design. The application field and resource utilization of each benchmark netlist are shown in Table II.

TABLE II
STATISTICS OF BENCHMARK NETLISTS

Name	Field	LUT/FF/BRAM/DSP/URAM	Total
blue-rdma	Network	119k/128k/125/0/0	248k
nvdla-1	DL	120k/113k/128/32/0	244k
nvdla-2	DL	143k/130k/128/52/0	288k
nvdla-3	DL	194k/158k/128/65/0	370k
nantucket-1	Crypto	93k/110k/0/576/16	207k
nantucket-2	Crypto	185k/221k/0/1152/32	417k
corundum	Network	74k/71k/238/0/45	149k
minimap-1	Genetics	185k/144k/10/280/0	342k
ispd-fpga02	Synthetic	100k/66k/100/100/0	166k
hardcamel-ntt	Crypto	88k/60k/0/1024/192	172k

C. Netlist Abstraction

In the netlist abstraction stage, we aim to conceal combinational cells and nets in the abstract hypergraph to achieve timing isolation between partitioned islands. However, experimental results reveal that clustering all connected combinational cells in the abstract hypergraph may result in extremely large clusters for certain benchmarks. To ensure balanced

island sizes in the subsequent stage of design partition, we have to leave some nets driven by combinational cells in the abstract hypergraph during net filtering. Specifically, we adopt two net filtering schemes during netlist abstraction: (1) **FF Only**: expose only nets driven by Flip Flop (FF); (2) **FF & LUT**: expose nets driven by either FF or Lookup Table (LUT). Evaluation of two schemes is shown in Table III, which reports total number of nodes in the abstract hypergraph and the size of the largest node. Specifically, the size of an abstract node is measured by the number of cells clustered in it and the ratio of this number to the total number of cells in the netlist. And this ratio is used to determine whether the abstract hypergraph is balanced. Empirically, we set the threshold ratio at 18% in our experiments. For each benchmark, we preferentially apply the first scheme. If it fails to produce balanced abstract hypergraph, we switch to the second one.

TABLE III
EVALUATION OF TWO NET FILTERING SCHEME

Name	FF Only		FF & LUT	
	#Node	Max Node	#Node	Max Node
blue-rdma	791	29.7k (11.98%)	70.1k	1.3k (0.53%)
nvdla-1	17.7k	29.1k (11.95%)	153.3k	3.9k (1.59%)
nvdla-2	18.0k	54.7k (18.98%)	181.0k	4.8k (1.65%)
nvdla-3	21.9k	46.0k (12.45%)	224.5k	3.9k (1.05%)
nantucket-1	44.8k	1.6k (0.76 %)	61.0k	907 (0.44%)
nantucket-2	89.5k	1.6k (0.38 %)	125.1k	903 (0.22%)
corundum	4.8k	16.8k (11.29%)	82.3k	1.5k (0.99%)
minimap	14.5k	223.2k (65.34%)	182.7k	23.8k (6.98%)
ispd-fpga02	13.1k	136.1k (81.88%)	156.8k	3.0k (1.79%)
hardcamel-ntt	4.4k	32.6k (18.92%)	52.6k	3.4k (1.97%)

Based on experimental results, we identify certain characteristics of benchmarks that can produce balanced abstract hypergraph under the first filtering scheme. From the perspective of netlist, these benchmarks exhibit a higher ratio of FF cells to the total number of cells. Additionally, they contain fewer high-fanout control signals, such as register enable signals, driven by combinational logic. From an architectural perspective, these benchmarks feature simple pipeline structures with minimal data interaction between different pipeline stages. Additionally, these benchmarks typically achieve higher working frequency than those that cannot produce balanced abstract hypergraphs.

D. Runtime Comparison

To evaluate the efficiency of our proposed flow, the runtime of each benchmark is compared with the standard physical design flow provided by the vendor Vivado [29]. We do not include other baselines because no current works of accelerating physical design are capable of handling synthesized netlists directly. For works [9] [11] also based on design-level parallelism, they are only applicable to HLS designs. For works [3] [1] [4] of algorithm-level parallelism, they only target one part of physical design, either routing or placement, and most of them are only applicable to synthetic benchmarks [35] with limited kinds of primitive resources. The runtime of physical design is highly related with timing constraints.

For fair comparisons, we set the same clock period constraint for both our flow and the standard Vivado flow, using the minimum value at which Vivado can achieve timing closure [22]. Specifically, for each benchmark, we perform placement and routing repeatedly with the constrained period decreasing in interval of 0.2ns to find the tightest clock constraint. The constrained clock period and runtime comparison of each benchmark are shown in Table IV.

TABLE IV
RUNTIME COMPARISON ON ALL BENCHMARKS

Name	Period (ns)	Vivado (s)	Our Flow (s)	Speedup
blue-rdma	3.6	1946	1195	1.63x
nvdla-1	5.2	1768	851	2.08x
nvdla-2	5.0	2204	1026	2.15x
nvdla-3	5.8	3244	1508	2.31x
nantucket-1	1.7	1458	801	1.82x
nantucket-2	1.8	3472	1570	2.21x
corundum	3.6	2316	912	2.54x
minimap	3.8	3462	1372	2.50x
ispd-fpga02	3.9	1970	928	2.12x
hardcaml-ntt	3.2	1580	1089	1.5x
Average	3.76	2342	1125	2.1x

The runtime breakdown of the overall flow and the final stage, parallel PnR & merge, is reported in Fig. 6 for benchmark **nvdla-3**. And similar runtime distributions are also observed on other benchmark netlists. The first two stages, including netlist abstraction and design partition, only account for about 2% of the total runtime. The majority of the runtime (about 97%) is consumed by the final stage, where we call the vendor tool Vivado to perform the physical design of islands and merge them together. In the final stage, the placement of anchor cells takes about 13% of the total runtime. And the remaining runtime of the last stage is primarily consumed by rerouting the boundary region and the parallel placement and routing of islands, each accounting for about 40%.

Noticeably, the step of rerouting of boundary region induces significant runtime overhead, which heavily undermines the speedup achieved through our design-level parallelism scheme. Even though only a few conflicted nets need to be rerouted, the vendor tool follows the same flow as routing the entire design, in which many steps are unnecessary and time-consuming. Rapidstream [9] proposes a custom partial router based on the open-source router RwRoute [25] to speedup the process of resolving conflicted nets, which may also be helpful to our flow. In addition to rerouting boundary, the runtime of parallel placement and routing of islands can also be further optimized by dividing the design into smaller pieces using a finer grid.

E. Frequency Degradation

The impact of our split-and-parallel flow on design performance is also evaluated. We compare the maximum achievable frequency of our flow with that of the standard Vivado flow. The detailed comparison results are shown in Fig. 7. Our flow introduces an average frequency loss of 6% on all ten benchmarks. For most designs, our flow ensures no or only trivial frequency degradation (less than 0.2ns increase

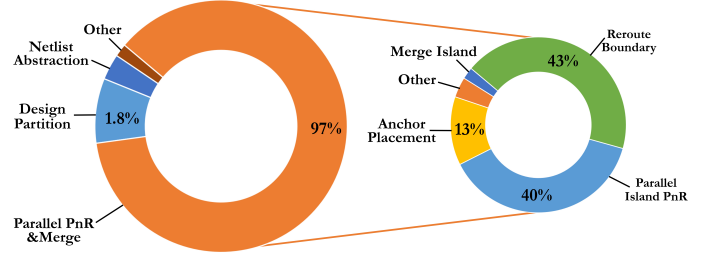


Fig. 6. Runtime breakdown of the overall flow and the stage of Parallel PnR & Merge stage on benchmark **nvdla-3**.

in critical path delay). These designs all adopt the **FF Only** filtering scheme during netlist abstraction, which ensures that partitioned islands are timing-independent from each other. While there is relatively significant frequency degradation (about 0.8ns increase in critical path delay) in several designs, such as ispd-fpga02 and hardcaml-ntt, which adopt the **FF & LUT** filtering scheme. This filtering scheme introduces inter-island timing paths, which impacts timing closure of the merged designs. Some potential methods to address this issue can be further explored in the future: 1) improve accuracy of external delay prediction to set more appropriate constraints on inter-island timing paths; 2) adopt timing-driven partition in min-cut island placement to reduce number of cuts on critical timing paths [24].

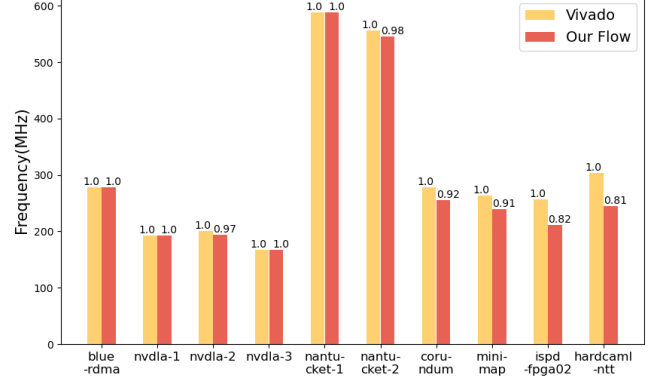


Fig. 7. Comparison of the achievable frequency between our flow and the standard Vivado flow.

V. CONCLUSION

In this paper, we extend the idea of design-level parallelism, originally applied to HLS designs, to more general scenarios where the physical design starts from the synthesized netlist. Based on this idea, we proposed an end-to-end physical design flow, which automatically partitions the synthesized netlist, parallelly places and routes each sub-netlist, and then merge them into the complete design. Experimental results show that our flow achieves an average speedup of 2.1x compared to standard Vivado flow, with only about 6% degradation in design performance.

REFERENCES

- [1] R. S. Rajarathnam, M. B. Alawieh, Z. Jiang, M. Iyer and D. Z. Pan, "DREAMPlaceFPGA: An Open-Source Analytical Placer for Large Scale Heterogeneous FPGAs using Deep-Learning Toolkit," 2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC), Taipei, Taiwan, 2022, pp. 300-306.
- [2] J. Mai, Y. Meng, Z. Di, and Y. Lin, "Multi-electrostatic fpga placement considering slicel-slicem heterogeneity and clock feasibility," in Proceedings of the 59th ACM/IEEE Design Automation Conference, pp. 649-654, 2022.
- [3] Y. Meng, W. Li, Y. Lin and D. Z. Pan, "elfPlace: Electrostatics-Based Placement for Large-Scale Heterogeneous FPGAs," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 41, no. 1, pp. 155-168, Jan. 2022.
- [4] T. Martin, D. Maarouf, G. Grewal and S. Areibi, "A High-Performance Routing Engine for Large-Scale FPGAs," 2024 34th International Conference on Field-Programmable Logic and Applications (FPL), Torino, Italy, 2024, pp. 53-59.
- [5] X. Zang, W. Lin, S. Lin, J. Liu, and E. F. Young, "An open-source fast parallel routing approach for commercial fpgas," in Proceedings of the Great Lakes Symposium on VLSI 2024, pp. 164-169, 2024.
- [6] Y. Zhou, P. Maidee, C. Lavin, A. Kaviani, and D. Stroobandt, "Rwroute: An open-source timing-driven router for commercial fpgas," ACM Transactions on Reconfigurable Technology and Systems (TRETS), vol. 15, no. 1, pp. 1-27, 2021.
- [7] M. Shen, G. Luo and N. Xiao, "Exploring GPU-Accelerated Routing for FPGAs," in IEEE Transactions on Parallel and Distributed Systems, vol. 30, no. 6, pp. 1331-1345, 1 June 2019.
- [8] K. E. Murray and V. Betz, "Tatum: Parallel Timing Analysis for Faster Design Cycles and Improved Optimization," 2018 International Conference on Field-Programmable Technology (FPT), Naha, Japan, 2018, pp. 110-117.
- [9] L. Guo, P. Maidee, Y. Zhou, C. Lavin, J. Wang, Y. Chi, W. Qiao, A. Kaviani, Z. Zhang, and J. Cong, "Rapidstream: parallel physical implementation of fpga hls designs," in Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 1-12, 2022.
- [10] D. Park, Y. Xiao, N. Magnezi and A. DeHon, "Case for Fast FPGA Compilation Using Partial Reconfiguration," 2018 28th International Conference on Field Programmable Logic and Applications (FPL), Dublin, Ireland, 2018, pp. 235-2353.
- [11] D. Park, Y. Xiao and A. DeHon, "Fast and Flexible FPGA Development using Hierarchical Partial Reconfiguration," 2022 International Conference on Field-Programmable Technology (ICFPT), Hong Kong, 2022, pp. 1-10.
- [12] Y. Xiao, S. T. Ahmed and A. DeHon, "Fast Linking of Separately-Compiled FPGA Blocks without a NoC," 2020 International Conference on Field-Programmable Technology (ICFPT), Maui, HI, USA, 2020, pp. 196-205.
- [13] Y. Xiao et al., "Reducing FPGA Compile Time with Separate Compilation for FPGA Building Blocks," 2019 International Conference on Field-Programmable Technology (ICFPT), Tianjin, China, 2019, pp. 153-161.
- [14] AMD-Xilinx. UltraScale Architecture Configuration User Guide (UG570). (2024). Accessed: Jan.8, 2025. [Online]. Available: <https://docs.amd.com/v/u/en-US/ug570-ultrascale-configuration>.
- [15] AMD-Xilinx. UltraScale Architecture Configurable Logic Block User Guide(UG574). (2017). Accessed: Jan.8, 2025. [Online]. Available: <https://docs.amd.com/v/u/en-US/ug574-ultrascale-clb>.
- [16] AMD-Xilinx. UltraFast Design Methodology Guide for Xilinx FPGAs and SoCs (UG949). (2024). Accessed: Jan.8, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/ug949-vivado-design-methodology>.
- [17] AMD-Xilinx. UltraScale Architecture Libraries Guide (UG974). (2024). Accessed: Jan.8, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/ug974-vivado-ultrascale-libraries>.
- [18] A. E. Caldwell, A. B. Kahng, and I. L. Markov, "Can recursive bisection alone produce routable placements?," in Proceedings of the 37th annual design automation conference, pp. 477-482, 2000.
- [19] J. A. Roy, D. A. Papa, S. N. Adya, H. H. Chan, A. N. Ng, J. F. Lu, and I. L. Markov, "Capo: robust and scalable open-source min-cut floorplacer," in Proceedings of the 2005 international symposium on Physical design, pp. 224-226, 2005.
- [20] C. Lavin and A. Kaviani, "RapidWright: Enabling Custom Crafted Implementations for FPGAs," 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Boulder, CO, USA, 2018, pp. 133-140.
- [21] C. Lavin and A. Kaviani, "Build your own domain-specific solutions with rapidwright: invited tutorial," in Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 14-22, 2019.
- [22] Z. Xiong, R. S. Rajarathnam and D. Z. Pan, "A Data-Driven, Congestion-Aware and Open-Source Timing-Driven FPGA Placer Accelerated by GPUs," 2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Orlando, FL, USA, 2024, pp. 115-125.
- [23] I. Bustany, G. Gasparyan, A. Gupta, A. B. Kahng, M. Kalase, W. Li and B. Pramanik, "The 2023 MLCAD FPGA Macro Placement Benchmark Design Suite and Contest Results", in Proceedings of the 2023 ACM/IEEE Workshop on Machine Learning for CAD, 2023.
- [24] I. Bustany, G. Gasparyan, A. B. Kahng, I. Koutis, B. Pramanik and Z. Wang, "An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design," 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD), San Francisco, CA, USA, 2023, pp. 1-9.
- [25] Y. Zhou, P. Maidee, C. Lavin, A. Kaviani, and D. Stroobandt, "Rwroute: An open-source timing-driven router for commercial fpgas," ACM Transactions on Reconfigurable Technology and Systems (TRETS), vol. 15, no. 1, pp. 1-27, 2021.
- [26] A. Samajdar, T. Garg, T. Krishna and N. Kapre, "Scaling the Cascades: Interconnect-Aware FPGA Implementation of Machine Learning Problems," 2019 29th International Conference on Field Programmable Logic and Applications (FPL), Barcelona, Spain, 2019, pp. 342-349.
- [27] A. Forencich, A. C. Snoeren, G. Porter and G. Papen, "Corundum: An Open-Source 100-Gbps Nic," 2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Fayetteville, AR, USA, 2020, pp. 38-46.
- [28] L. Guo, J. Lau, Z. Ruan, P. Wei and J. Cong, "Hardware Acceleration of Long Read Pairwise Overlapping in Genome Sequencing: A Race Between FPGA and GPU," 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), San Diego, CA, USA, 2019, pp. 127-135.
- [29] AMD-Xilinx, Vivado ML Edition 2023.2, 2023. [Online]. Available: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>.
- [30] Nvidia, Hardware Manual of NVIDIA Deep Learning Accelerator (NVDLA), 2017. [Online]. Available: <https://github.com/nvdl/hw>.
- [31] Google, OR-Tools: an open-source, fast and portable software suite for solving combinatorial optimization problems, 2024. [Online]. Available: <https://github.com/google/or-tools>.
- [32] Datenlord, blue-rdma, 2024. [Online]. Available: <https://github.com/datenlord/blue-rdma>.
- [33] Zprize Contest, hardcaml-ntt, 2022. [Online]. Available: <https://github.com/z-prize/2022-entries/tree/main/open-division/prize2-ntt/hardcaml>.
- [34] Zprize Contest, nantucket, 2022. [Online]. Available: <https://github.com/z-prize/2022-entries/tree/main/open-division/prize2-ntt/supranational>.
- [35] ISPD Contest, "ISPD 2016: Routability-Driven FPGA Placement Contest", Accessed: Jan. 13, 2025. [Online]. Available: https://www.ispd.cc/contests/16/ispd2016_contest.html.